1. En orientación a objetos, ¿qué es la **herencia** y su relación con "A es-un B"?. Explica las dos implicaciones principales: (1) **compatibilidad de tipos** y (2) **herencia de estado y comportamiento**. Pon un ejemplo en Java muy sencillo, donde un `Soldado` tiene un `nombre` (privado) y un método `saludar()` que muestra su nombre. Hay dos subtipos: un `Artillero` , que es capaz de disparar cohetes y un `Zapador` que pone minas, ambos heredan el atributo nombre y la capacidad de saludar. Además, y de forma específica, el artillero tiene un número de cohetes y el zapador un número de minas, accesibles mediante "getters" específicos. Respecto a la compatibilidad de tipos, aprovechémosla: crea un array de `Soldado` , mete varios de distinto tipo (son todos compatibles con `Soldado`). Recórrela y que todos te saluden.

Respuesta

La herencia en orientación a objetos es un mecanismo que permite que una clase (subclase) adquiera las propiedades y comportamientos de otra clase (superclase). Esto se relaciona con el concepto "A es-un B" porque establece una relación jerárquica donde la subclase es un tipo especializado de la superclase. Por ejemplo, un `Artillero` y un `Zapador` son tipos de `Soldado` .

1. **Compatibilidad de tipos**: Gracias a la herencia, un objeto de una subclase puede ser tratado como si fuera de la superclase. Esto permite almacenar objetos de diferentes subclases en una misma estructura, como un array de `Soldado` .
2. **Herencia de estado y comportamiento**: Las subclases heredan atributos y métodos de la superclase, lo que evita la duplicación de código. Además, pueden agregar o sobrescribir comportamientos específicos.

Ejemplo en Java:

```java
class Soldado {
    private String nombre;

    public Soldado(String nombre) {
        this.nombre = nombre;
    }

    public void saludar() {
        System.out.println("Hola, soy " + nombre);
    }
}

class Artillero extends Soldado {
    private int cohetes;

    public Artillero(String nombre, int cohetes) {
        super(nombre);
        this.cohetes = cohetes;
    }

    public int getCohetes() {
        return cohetes;
    }
}

class Zapador extends Soldado {
    private int minas;

    public Zapador(String nombre, int minas) {
        super(nombre);
        this.minas = minas;
    }

    public int getMinas() {
        return minas;
    }
}

public class Main {
    public static void main(String[] args) {
        Soldado[] soldados = {
            new Artillero("Juan", 5),
            new Zapador("Pedro", 3)
```

```
        };

        for (Soldado s : soldados) {
            s.saludar();
        }
    }
}
```

2. Al crear los soldados concretos, ¿cuántos constructores se ejecutan y en qué orden? ¿Qué significa `super` dentro de un constructor? Si la clase base no tiene visible el constructor sin parámetros, ¿debo llamar a `super` siempre?

Respuesta

Al crear un objeto de una subclase, se ejecutan los constructores de la subclase y de la superclase. El orden es siempre de la superclase a la subclase. Esto asegura que los atributos heredados sean inicializados correctamente antes de los específicos de la subclase.

El uso de `super` dentro de un constructor permite llamar explícitamente al constructor de la superclase. Esto es necesario cuando la superclase no tiene un constructor sin parámetros o cuando se desea inicializar atributos heredados con valores específicos.

Si la clase base no tiene un constructor sin parámetros, es obligatorio llamar a `super` con los argumentos requeridos. De lo contrario, el código no compilará.

Ejemplo:

```java
class Soldado {
    private String nombre;

    public Soldado(String nombre) {
        this.nombre = nombre;
    }
}

class Artillero extends Soldado {
    public Artillero(String nombre) {
        super(nombre); // Obligatorio porque Soldado no tiene constructor sin parámetros
    }
}
```

3. Respecto a los objetos de subclases en memoria, los atributos privados de la superclase, ¿forman parte de una instancia de la subclase en memoria? En caso afirmativo ¿implica que se puedan usar desde el código de la subclase? Explícalo con el ejemplo de `Soldado` y alguna de sus subclases.

Respuesta

Sí, los atributos privados de la superclase forman parte de la instancia de la subclase en memoria. Sin embargo, no son accesibles directamente desde el código de la subclase. Esto se debe a que el modificador `private` restringe el acceso únicamente a la clase donde se declara el atributo.

Para acceder a estos atributos desde la subclase, se deben usar métodos públicos o protegidos definidos en la superclase. Por ejemplo:

```java
class Soldado {
    private String nombre;

    public Soldado(String nombre) {
        this.nombre = nombre;
    }

    protected String getNombre() {
        return nombre;
    }
}

class Zapador extends Soldado {
    public Zapador(String nombre) {
        super(nombre);
    }

    public void ponerMina() {
        System.out.println(getNombre() + " está poniendo una mina.");
    }
}
```

En este caso, el método `getNombre` permite acceder al atributo privado `nombre` desde la subclase `Zapador`.

4. ¿Qué implica en términos de **extensibilidad** de código el hecho de que sean compatibles a nivel de tipos? Ilustra esto añadiendo un nuevo tipo de `Soldado` y demostrando que el código para pedir el saludo a todos los soldados no se modifica.

Respuesta

La compatibilidad a nivel de tipos implica que se pueden agregar nuevas subclases sin necesidad de modificar el código que utiliza la superclase. Esto mejora la extensibilidad, ya que el sistema puede manejar nuevos tipos de objetos sin cambios significativos.

Por ejemplo, si se añade un nuevo tipo de `Soldado`, como un `Francotirador`, el código que recorre un array de `Soldado` para saludarlos no necesita modificaciones:

```
class Francotirador extends Soldado {
    public Francotirador(String nombre) {
        super(nombre);
    }
}

public class Main {
    public static void main(String[] args) {
        Soldado[] soldados = {
            new Artillero("Juan", 5),
            new Zapador("Pedro", 3),
            new Francotirador("Luis")
        };

        for (Soldado s : soldados) {
            s.saludar();
        }
    }
}
```

En este caso, el código para recorrer el array no cambia, pero se pueden manejar nuevos tipos de soldados.

5. En Java, cuando trabajo con referencias y herencia. ¿Puedo tener una referencia del supertipo que apunte a objetos reales de un subtipo? ¿Puedo invocar con la referencia del supertipo a métodos públicos del subtipo? ¿En qué consiste el "upcasting" y el "downcasting"? ¿Qué es el `instanceof`? Pon un ejemplo de recorrido de un array de `Soldado`,

comprobando que, si el objeto real es un `Artillero`, solicite el número de cohetes que tiene y los imprima.

Respuesta

En Java, es posible tener una referencia del supertipo que apunte a objetos reales de un subtipo. Esto se conoce como **upcasting** y ocurre de manera implícita. Sin embargo, con una referencia del supertipo solo se pueden invocar los métodos definidos en la superclase, incluso si el objeto real pertenece a un subtipo.

Para acceder a métodos específicos del subtipo, se debe realizar un **downcasting**, que es convertir explícitamente la referencia del supertipo al subtipo. Antes de realizar un downcasting, es recomendable usar el operador `instanceof` para verificar el tipo real del objeto y evitar errores en tiempo de ejecución.

Ejemplo:

```java
public class Main {
    public static void main(String[] args) {
        Soldado[] soldados = {
            new Artillero("Juan", 5),
            new Zapador("Pedro", 3)
        };

        for (Soldado s : soldados) {
            s.saludar();
            if (s instanceof Artillero) {
                Artillero a = (Artillero) s; // Downcasting
                System.out.println("Cohetes: " + a.getCohetes());
            }
        }
    }
}
```

En este ejemplo, se recorre un array de `Soldado` y, si el objeto real es un `Artillero`, se realiza un downcasting para acceder al método `getCohetes`.

6. Respecto a la ocultación de información y herencia, ¿qué significa acceso "protegido" de métodos y/o atributos? ¿Cómo se implementa en Java? Pon un ejemplo de uso de en la clase `Soldado` para que su nombre sea protegido y pueda usarse en el método de poner bombas del `Zapador` .

Respuesta

El acceso **protegido** (modificador `protected`) en Java permite que un atributo o método sea accesible desde:

1. La misma clase donde se declara.
2. Clases del mismo paquete.
3. Subclases, incluso si están en paquetes diferentes.

Esto es útil en herencia, ya que permite que las subclases accedan a ciertos atributos o métodos sin exponerlos completamente al resto del programa.

Ejemplo:

```java
class Soldado {
    protected String nombre;

    public Soldado(String nombre) {
        this.nombre = nombre;
    }
}

class Zapador extends Soldado {
    public Zapador(String nombre) {
        super(nombre);
    }

    public void ponerMina() {
        System.out.println(nombre + " está poniendo una mina.");
    }
}
```

En este caso, el atributo `nombre` es protegido, lo que permite que la subclase `Zapador` lo utilice directamente en su método `ponerMina`.

7. En los lenguajes orientados a objetos ¿hay una **clase base** para todos los objetos? ¿Ocurre en todos los lenguajes? ¿Qué ocurre en Java?

Respuesta

En muchos lenguajes orientados a objetos, como Java, existe una clase base para todos los objetos. En Java, esta clase se llama `Object`. Todas las clases en Java heredan implícitamente de `Object`, ya sea directa o indirectamente. Esto significa que cualquier objeto en Java tiene acceso a los métodos definidos en `Object`, como `toString()`, `equals()` y `hashCode()`.

Sin embargo, no todos los lenguajes orientados a objetos tienen una clase base común. Por ejemplo, en C++ no existe una clase base universal; las clases pueden definirse sin heredar de ninguna otra.

La existencia de una clase base común en Java facilita la manipulación genérica de objetos, ya que se pueden tratar como instancias de `Object` cuando sea necesario.

Ejemplo:

```
Object obj = new String("Hola");
System.out.println(obj.toString());
```

En este caso, aunque `obj` es de tipo `Object`, puede almacenar cualquier objeto, como una instancia de `String`.

8. ¿Qué es la "herencia múltiple"? ¿Existe en Java herencia múltiple?

Respuesta

La **herencia múltiple** es un mecanismo en el que una clase puede heredar de más de una superclase. Esto permite que una clase combine comportamientos y atributos de múltiples clases base. Sin embargo, este enfoque puede generar problemas como el **"problema del diamante"**, donde un método o atributo puede ser heredado de múltiples superclases, causando ambigüedad.

En Java, no se permite la herencia múltiple de clases para evitar estos problemas. Una clase solo puede heredar de una única superclase. Sin embargo, Java permite la herencia múltiple de interfaces. Una clase puede implementar múltiples interfaces, lo que proporciona una forma de lograr polimorfismo sin los problemas asociados con la herencia múltiple de clases.

Ejemplo de herencia múltiple con interfaces:

```java
interface Volador {
    void volar();
}

interface Nadador {
    void nadar();
}

class SuperHeroe implements Volador, Nadador {
    public void volar() {
        System.out.println("Estoy volando.");
    }

    public void nadar() {
        System.out.println("Estoy nadando.");
    }
}
```

En este ejemplo, la clase `SuperHeroe` implementa dos interfaces, logrando una forma de herencia múltiple sin ambigüedades.

9. Las excepciones en los lenguajes orientados a objetos son objetos. Por tanto, se pueden crear excepciones personalizadas. Pon un ejemplo en Java de una excepción personalizada (`UsuarioNoEncontradoException`), que sea *no controlada* y que además este compuesto con un `Usuario` , para saber qué `Usuario` dio el problema. Permite además que se pueda incluir la causa, es decir, sobrecarga el constructor para tener una versión que permita añadir la causa subyacente.

Respuesta

En Java, las excepciones son objetos que heredan de la clase `Throwable` . Se pueden crear excepciones personalizadas para manejar casos específicos. Una excepción *no controlada* (unchecked) hereda de `RuntimeException` . A continuación, se muestra un ejemplo de una excepción personalizada `UsuarioNoEncontradoException` que incluye un objeto `Usuario` y permite añadir una causa subyacente:

```java
class Usuario {
    private String nombre;

    public Usuario(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }
}

class UsuarioNoEncontradoException extends RuntimeException {
    private Usuario usuario;

    public UsuarioNoEncontradoException(Usuario usuario) {
        super("Usuario no encontrado: " + usuario.getNombre());
        this.usuario = usuario;
    }

    public UsuarioNoEncontradoException(Usuario usuario, Throwable causa) {
        super("Usuario no encontrado: " + usuario.getNombre(), causa);
        this.usuario = usuario;
    }

    public Usuario getUsuario() {
        return usuario;
    }
}

public class Main {
    public static void main(String[] args) {
        Usuario usuario = new Usuario("Juan");
        throw new UsuarioNoEncontradoException(usuario);
    }
}
```

En este ejemplo, la excepción personalizada incluye información sobre el usuario que causó el problema y permite incluir una causa subyacente.

10. Herencia vs. Composición. Se dice que no se debe emplear herencia simplemente por reutilizar código, es decir, que si quiero reutilizar código simplemente, no debo pensar en herencia como primera opción ¿por qué?

Respuesta

La herencia no debe usarse únicamente para reutilizar código porque introduce una relación jerárquica entre clases que puede ser innecesaria o inapropiada. La herencia implica que la subclase es un tipo especializado de la superclase (relación "es-un"), lo cual no siempre es cierto cuando solo se busca reutilizar código.

Además, la herencia puede acoplar fuertemente las clases, dificultando su mantenimiento y evolución. Si se realizan cambios en la superclase, estos pueden afectar a todas las subclases, incluso si no están directamente relacionadas con el cambio.

En lugar de herencia, se recomienda usar composición para reutilizar código. La composición permite que una clase incluya instancias de otras clases y delegue responsabilidades, lo que resulta en un diseño más flexible y modular.

11. Herencia vs. Composición. Se dice que se debe *"favorecer la composición frente a la herencia"*, ¿por qué?

Respuesta

Se recomienda favorecer la composición frente a la herencia porque la composición ofrece mayor flexibilidad y reduce el acoplamiento entre clases. En la composición, una clase puede incluir instancias de otras clases y delegarles responsabilidades, lo que permite cambiar o extender el comportamiento sin afectar a la jerarquía de clases.

Por otro lado, la herencia introduce una relación jerárquica rígida que puede ser difícil de modificar o extender. Además, la herencia puede llevar a problemas como el "problema del diamante" (en

lenguajes con herencia múltiple) o la propagación de cambios no deseados desde la superclase a las subclases.

La composición también fomenta el principio de diseño "programar para una interfaz, no para una implementación", lo que facilita la reutilización y el mantenimiento del código.

12. Herencia vs. Composición. Se dice que la *"herencia rompe la encapsulación"*, ¿a qué se refiere esto?

Respuesta

La afirmación de que "la herencia rompe la encapsulación" se refiere a que, en una relación de herencia, las subclases tienen acceso directo a los detalles internos de la superclase, especialmente si los atributos o métodos están protegidos (`protected`). Esto puede exponer detalles de implementación que deberían permanecer ocultos, violando el principio de encapsulación.

Por ejemplo, si una subclase depende de un atributo protegido de la superclase y este atributo cambia, la subclase puede verse afectada, lo que dificulta el mantenimiento y la evolución del código.

En contraste, la composición respeta mejor la encapsulación porque las clases interactúan a través de interfaces bien definidas, sin acceder directamente a los detalles internos de otras clases.

13. Pongamos un ejemplo de dos alternativas para lo mismo. Tenemos un `Estudiante` y un `Trabajador` , ambos tienen datos en común: el DNI y el nombre. Modelemos esto de dos formas: uno por herencia, con una superclase `Persona` , y otro con composición, con una clase `DatosPersonales` . Se debe recibir una

instancia de `DatosPersonales` en el constructor de la clase `Estudiante` y `Trabajador`.

Respuesta

Con herencia:

```
class Persona {
    private String dni;
    private String nombre;

    public Persona(String dni, String nombre) {
        this.dni = dni;
        this.nombre = nombre;
    }

    public String getDni() {
        return dni;
    }

    public String getNombre() {
        return nombre;
    }
}

class Estudiante extends Persona {
    public Estudiante(String dni, String nombre) {
        super(dni, nombre);
    }
}

class Trabajador extends Persona {
    public Trabajador(String dni, String nombre) {
        super(dni, nombre);
    }
}
```

Con composición:

```java
class DatosPersonales {
    private String dni;
    private String nombre;

    public DatosPersonales(String dni, String nombre) {
        this.dni = dni;
        this.nombre = nombre;
    }

    public String getDni() {
        return dni;
    }

    public String getNombre() {
        return nombre;
    }
}

class Estudiante {
    private DatosPersonales datos;

    public Estudiante(DatosPersonales datos) {
        this.datos = datos;
    }

    public DatosPersonales getDatos() {
        return datos;
    }
}

class Trabajador {
    private DatosPersonales datos;

    public Trabajador(DatosPersonales datos) {
        this.datos = datos;
    }

    public DatosPersonales getDatos() {
        return datos;
    }
}
```

En el enfoque de composición, `Estudiante` y `Trabajador` delegan la gestión de los datos personales a la clase `DatosPersonales` , lo que permite mayor flexibilidad y reutilización del código.